having to import models from other modules and solve potential circular import problems. If the relationship is between models in the same file, you can use the class itself. If the relationship is between models in different files (and likely different apps), you should use the string specifier.

The second argument to the `ForeignKey` field is the `on_delete` parameter. In older versions of Django, this was optional, but it is now a required argument. This parameter specifies how Django is to behave if the associated object gets deleted. The choices for `on_delete` are a series of constants defined in the `django.db.models` module. The two most common choices are

- `CASCADE`—The associated object also gets deleted.
- `SET_NULL`—Set the field to `NULL`.

Other choices are available as well, including being able to define a function called when the model gets loaded. See http://mng.bz/679R for a complete listing. Up until now, all the fields you defined were required fields. To use `on_delete=SET_NULL`, the field must be allowed to contain a `NULL` value. To do this, pass `null=True` to the field constructor. Without it, the default of `null=False` is assumed.

Django comes with a web-based tool for managing the objects in your database, called the Django Admin. You can create objects in the tool, including specifying inter-object relationships. The Django Admin allows you to define whether the UI permits a relationship to be empty. To give you fine-grained control, Django distinguishes between `NULL` in the database and an empty value in the Django Admin. To allow empty values, you pass `blank=True` to the field constructor. You'll learn more about the Django Admin and these controls in chapter 5.

Let's add two new models to RiffMates: one for our venues and one for the rooms in those venues. Both the venues and the rooms need to have names, and additionally, the room needs to specify its associated venue. The `Room` model uses a `ForeignKey` field to create this relationship. If a venue gets removed from the database, it doesn't make sense to keep the associated rooms, so the `on_delete` value for the `Room` is `CASCADE`. Open RiffMates/bands/models.py, and add the `Venue` and `Room` models shown in the following listing.

Listing 4.13 `Venue` and `Room` models and their one-to-many relationship

```
# RiffMates/bands/models.py
...
class Venue(models.Model):
    name = models.CharField(max_length=20)

    def __str__(self):
        return f"Venue(id={self.id}, name={self.name})"

class Room(models.Model):
    name = models.CharField(max_length=20)
    venue = models.ForeignKey(Venue, on_delete=models.CASCADE)

    def __str__(self):
        return f"Room(id={self.id}, name={self.name})"
```

Foreign key relationship between Venue and Room